

```
loop-do-while (condition body...) ;;
Syntax

(let ((x 0)
      (sum 0))
  ;; sum of 1..4
  (loop-do-while (and (> x 0) (< x 5))
    (setq sum (+ sum x))
    (setq x (1+ x)))
  sum)
10
```

The *loop-until* construct repeatedly evaluates the body of the code until the condition becomes true. For example:

```
loop-until (condition body...) ;;
Syntax

(let ((x 0)
      (sum 0))
  ;; sum of 0..4
  (loop-until (= x 5)
    (setq sum (+ sum x))
    (setq x (1+ x)))
  sum)
10
```

The *loop-for-each* construct takes three arguments – a var, a list and a body. The var represents an element in the list for the iteration. In the following example, the sum of numbers from 1 to 5 is calculated.

```
loop-for-each (var list body...) ;;
Syntax

(let ((sum 0))
  (loop-for-each x (list 1 2 3 4 5)
    (setq sum (+ sum x)))
  sum)
15
```

A couple of constructs are available to break or continue execution within a loop. The *loop-break* construct breaks out of the innermost loop. For example:

```
loop-break () ;; Syntax

(let ((sum 0))
```

```
;; sum 1..5
(loop-for-each x (list 1 2 3 4 5 6)
  (setq sum (+ sum x))
  (when (= x 5)
    (loop-break)))
sum)
15
```

The *loop-continue* construct will skip the rest of the current *loop-while*, *loop-do-while* or *loop-for-each* block and will proceed to the next iteration in the loop. In the following example, the list is iterated for elements between 1 and 6, and is skipped when the iteration matches the element 2.

```
loop-continue () ;; Syntax

(let ((sum 0))
  ;; sum the numbers 1, 3, 5
  (loop-for-each x (list 1 2 3 4 5 6)
    (when (= x 2)
      (loop-continue))
    (setq sum (+ sum x)))
  sum)
19
```

The *loop.el* library has unit tests included in the source code, which you can run to validate the defined constructs. In order to run the tests, you need to first clone the source repository using the following commands:

```
$ git clone https://github.com/Wilfred/loop.el
Cloning into 'loop.el'...
remote: Enumerating objects: 232, done.
remote: Total 232 (delta 0), reused 0 (delta 0), pack-reused 232
Receiving objects: 100% (232/232), 31.29 KiB | 801.00 KiB/s, done.
Resolving deltas: 100% (117/117), done.
```

If you do not have Cask, install it using the instructions provided in the README file at <https://github.com/cask/cask>.

You can then change the directory into the cloned *loop.el* folder, and run *cask install*. This will locally install the required dependencies for running

the tests.

```
$ cd loop.el/
$ cask install
Loading package information... Select coding system (default utf-8):
done
Package operations: 3 installs, 0 removals
- Installing [ 1/3] undercover (latest)... done
- Installing [ 2/3] ert-runner (latest)... done
- Installing [ 3/3] f (latest)... already present
```

A *Makefile* exists in the top-level directory, the contents of which are provided below for reference:

```
$ cat Makefile
CASK ?= cask
EMACS ?= emacs

all: test
test: unit

unit:
    ${CASK} exec ert-runner

install:
    ${CASK} install
```

You can now simply run *make test* at the shell prompt to execute the tests as shown below:

```
$ make test
cask exec ert-runner
.....

Ran 17 tests in 0.001 seconds
```

Readers are encouraged to go through the README file at <https://github.com/Wilfred/loop.el> for more information. **END** 

 By: Shakthi Kannan

The author is a free software enthusiast who blog at shaktimaan.com.